Data Management for Machine Learning – Assignment 2

End-to-End Data Management Pipeline for a Recommendation System

Name / ID: Abinaya R | 2025ag05178@wilp.bits-pilani.ac.in

GitHub repository: https://github.com/ABINAYARAGHUNATH/recommendation-pipeline

1. Overview and Architecture

A reproducible, modular pipeline for a movie recommendation system on MovieLens 100K (943 users, 1,682 movies, 100,000 ratings): ingestion, validation, preparation, transformation, feature management, model training, evaluation and orchestration. Each stage is a Python module that reads from and writes to disk, all settings live in one configuration file, and a single Prefect flow runs the stages in dependency order.

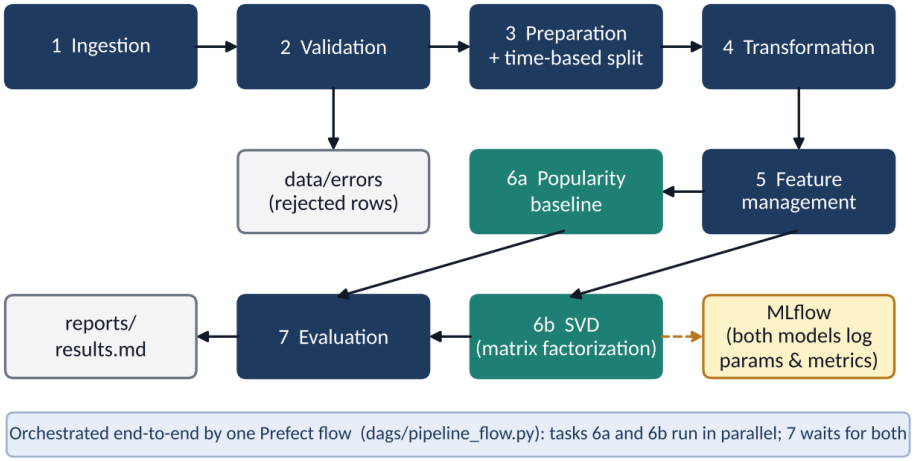


Figure 1. Pipeline architecture. Stages 6a and 6b run in parallel; the dashed arrow is experiment tracking.

2. Pipeline Stages and Design Decisions

Ingestion. Users, items and interactions are saved unchanged as CSV in data/raw/. Five faulty rows are appended on purpose (two copies, one rating of 9, one unknown user ID, one negative timestamp), giving 100,005 interactions; the download is skipped if the data already exists, so re-runs are safe.

Validation. Interactions are checked for missing values and wrong types, duplicate user–item pairs, ratings outside 1–5, invalid timestamps, and user or item IDs missing from the reference tables. Failing rows go to data/errors/ with the reason attached instead of being silently dropped, and a summary is written to reports/validation_report.json.

Preparation. Valid rows are de-duplicated and sorted by time, and attributes are standardized (median-imputed age, upper-cased gender, parsed release dates). The split is time-based, not random, so the model never learns from the future: training is the first 80% of timestamps (80,003 rows, up to 7 March 1998) and the rest is test. Test rows whose user or item is absent from training are dropped to avoid cold start, which shrinks the test set from 19,997 to 2,860 rows.

Transformation. From training data only (no leakage): a user activity summary, item popularity and a sparse user–item rating matrix (751 × 1,616, 6.59% dense).

Feature management. One function, build_features(), creates user, item and interaction features, so the same logic serves training and prediction. They are documented in reports/feature_catalog.md: user average rating, activity count, recency, preferred genre and demographics; item popularity, average rating and genres; interaction rating and recency.

Training and tracking. A popularity baseline (most-rated items the user has not seen) and a personalized truncated-SVD recommender (20 latent factors on the mean-centred user–item matrix, seed 42). Parameters, metrics and model files are logged to MLflow.

Orchestration. dags/pipeline_flow.py is a Prefect flow of eight tasks: ingest → validate → prepare → transform → features → (train popularity ‖ train SVD) → evaluate. A run completes in about 5 seconds (Section 7).

3. Tools Used and Rationale

- **pandas, SciPy, scikit-learn, Parquet:** data processing, sparse matrices, SVD, compact typed intermediate files.
- **Plain pandas validation rules:** no extra dependency; each rule yields an error dataset with a reason per rejected row.
- **MLflow:** tracks parameters, metrics and artifacts and compares runs side by side.
- **Prefect:** Python-native orchestration with dependencies, parallel tasks and a UI; lighter to run locally than Airflow.
- **Reproducibility:** one config file, fixed seed, idempotent ingestion, file hand-off between stages, requirements.txt and README instructions.

4. Data Quality Findings

Validation processed 100,005 interactions: 100,000 passed and 5 were quarantined in data/errors/interactions_errors.csv. The counts add up to 7 because a row can fail more than one check (the rating-9 and negative-timestamp rows are also duplicate pairs). The faulty rows were injected to demonstrate the checks.

Check failed	Flags	Meaning
Duplicate user–item pair	4	Same (user_id, item_id) appears more than once; first occurrence kept
Invalid rating	1	Rating outside the allowed range 1–5 (value 9)
Referential integrity (orphan user)	1	user_id not present in the users table (value 99999)
Invalid timestamp	1	Non-positive timestamp (value −5)

5. Models and Results

K = 10; an item is relevant if the user rated it 4 or higher in the test period. RMSE and MAE use the test ratings (the popularity model predicts each item's mean rating). Best value per metric in bold.

Model	P@10	R@10	NDCG@10	Coverage	Diversity	RMSE	MAE
Popularity (baseline)	0.1105	0.0860	0.1379	0.0458	**0.7486**	**1.0365**	**0.8331**
SVD (personalized)	**0.1158**	**0.1159**	**0.1755**	**0.1510**	0.7242	1.1507	0.9254

- **SVD ranks better:** recall rises from 0.086 to 0.116 and NDCG from 0.138 to 0.176; precision improves only slightly (0.1105 to 0.1158).
- **Coverage is 3.3 times higher** (15.1% vs 4.6% of the catalogue): SVD recommends a much wider set of movies.
- **Popularity wins on diversity and rating error** (0.749 vs 0.724; lower RMSE/MAE): truncated SVD treats unrated cells as zeros, so its scores suit ranking, not calibrated rating prediction.
- **Caution:** the test set is small (2,860 interactions), so the precision gap is indicative, not conclusive.

6. Limitations and Future Work

- **Small test set.** Users rate in bursts, so many users in the last 20% of the timeline never appear earlier and are removed (only 751 of 943 users and 1,616 of 1,682 movies are in training). A per-user time split would give a larger, more representative test set.
- **Features not yet consumed.** The feature tables are generated and catalogued, but both models use the interaction matrix and popularity statistics; the features are meant for a hybrid or learning-to-rank model.
- **Model quality.** Truncated SVD ignores missing ratings (ALS, Surprise or implicit would do better); no cold-start handling; offline evaluation only.
- **Future work:** scheduled Prefect deployments, data-drift monitoring, Pandera or Great Expectations rules, a feature store such as Feast.

7. Execution Evidence: Orchestration and Every Pipeline Stage

Orchestration (Stage 8). Successful Prefect run of the full flow (Figure 2); the experiment-tracking view follows the stage screenshots (Figure 3).

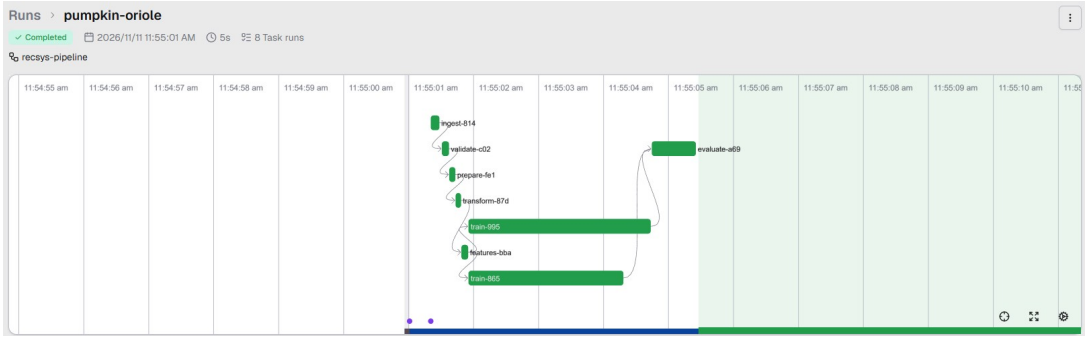


Figure 2. Prefect flow run "pumpkin-oriole": Completed, 8 task runs, about 5 s (task timeline).

Stages 1–7, each executed from the project root in the virtual environment (command and output shown):

Stage 1: Ingestion. 943 users, 1,682 items, 100,005 interactions loaded (incl. 5 injected faulty rows).

```
(venv) PS C:\Users\admin\recommendation-pipeline> python -c "from src.utils import load_config; from src.ingestion import ingest; ingest.run(load_config())"
Ingested: 943 users, 1682 items, 100005 interactions
(venv) PS C:\Users\admin\recommendation-pipeline>
```

Stage 2: Validation. 5 of 100,005 rows quarantined: duplicate 4, invalid rating 1, orphan user 1, invalid timestamp 1.

```
(venv) PS C:\Users\admin\recommendation-pipeline> python -c "from src.utils import load_config; from src.validation import validate; validate.run(load_config())"
{'total_rows': 100005, 'valid_rows': 100000, 'error_rows': 5, 'by_reason': {'duplicate': 4, 'invalid_rating': 1, 'orphan_user': 1, 'invalid_timestamp': 1}}
(venv) PS C:\Users\admin\recommendation-pipeline>
```

Stage 3: Preparation. Time-based split: 80,003 train rows (to 1998-03-07 02:21:09), 2,860 test rows (from 02:21:37).

```
(venv) PS C:\Users\admin\recommendation-pipeline> python -c "from src.utils import load_config; from src.preparation import prepare; prepare.run(load_config())"
Train: 80003 rows | Test: 2860 rows
Train ends: 1998-03-07 02:21:09
Test starts: 1998-03-07 02:21:37
(venv) PS C:\Users\admin\recommendation-pipeline>
```

Stage 4: Transformation. User summary (751 users), item popularity (1,616 items), 751 x 1,616 matrix, 6.59% dense.

```
(venv) PS C:\Users\admin\recommendation-pipeline> python -c "from src.utils import load_config; from src.transformation import transform; transform.run(load_config())"
User summary: 751 users
Item popularity: 1616 items
Matrix: 751 users x 1616 items, density 6.59%
(venv) PS C:\Users\admin\recommendation-pipeline>
```

Stage 5: Feature management. User (751 x 9), item (1,616 x 23) and interaction (80,003 x 5) feature tables created.

```
(venv) PS C:\Users\admin\recommendation-pipeline> python -c "from src.utils import load_config; from src.features import build_features; build_features.run(load_config())"
User features: (751, 9) | Item features: (1616, 23) | Interaction features: (80003, 5)
   user_id  user_avg_rating  user_activity_count  days_since_last_interaction preferred_genre
0        1         3.611321                  265                     5.837280          Drama
1        2         3.709677                   62                     2.974873          Drama
2        3         2.314286                   35                     0.000000       Thriller
3        5         2.874286                  175                   116.184850         Comedy
4        6         3.635071                  211                    63.888484          Drama
(venv) PS C:\Users\admin\recommendation-pipeline>
```

Stage 6: Model training. Popularity and SVD models trained and saved; both runs logged to MLflow.

```
(venv) PS C:\Users\admin\recommendation-pipeline> python -c "from src.utils import load_config; from src.training import train; c=load_config(); train.run(c,'popularity'); train.run(c,'svd')"
Trained popularity -> models/popularity.pkl
Trained svd -> models/svd.pkl
(venv) PS C:\Users\admin\recommendation-pipeline>
```

Stage 7: Model evaluation. P@10, R@10, NDCG@10, coverage, diversity, RMSE and MAE for both models.

```
(venv) PS C:\Users\admin\recommendation-pipeline> python -c "from src.utils import load_config; from src.evaluation import evaluate; evaluate.run(load_config())"
      model  precision@10  recall@10  ndcg@10  coverage  diversity   rmse    mae
popularity        0.1105     0.0860   0.1379    0.0458     0.7486 1.0365 0.8331
       svd        0.1158     0.1159   0.1755    0.1510     0.7242 1.1507 0.9254
(venv) PS C:\Users\admin\recommendation-pipeline>
```

Experiment tracking. Both models logged to MLflow and compared side by side.

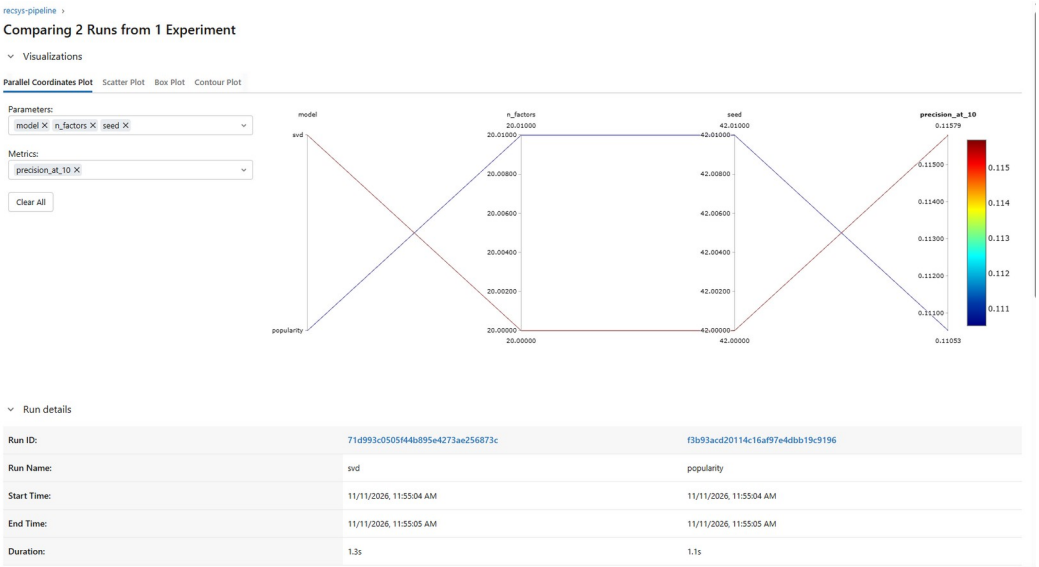


Figure 3. MLflow comparison of the svd and popularity runs (parameters, metric and run details).